UNIT 5 : GRAPH

Title: Apply DFS to a Computer Network Graph to Identify All Computers Reachable from a Selected Computer

Aim :

To implement the Depth First Search (DFS) algorithm on a computer network graph to identify all computers reachable from a selected computer using C++.

Problem Statement :

To implement the Depth First Search (DFS) algorithm using an adjacency list to represent a computer network as a graph, where computers are vertices and network connections are edges. The program accepts a starting computer as input, traverses the graph using DFS, and displays all computers reachable from the selected computer, identifying disconnected computers that cannot be reached.

Objectives :

1) To understand the concept of graphs and their applications in computer networks.
2) To represent a computer network using a graph.
3) To implement the Depth First Search (DFS) traversal algorithm.
4) To identify all computers reachable from a selected computer.
5) To understand how graph traversal helps in network connectivity analysis.

Theory :

Graph

A graph is a non-linear data structure consisting of vertices and edges. Vertices represent objects, while edges represent connections between those objects.

In a computer network, each computer can be represented as a vertex, and a communication link between two computers can be represented as an edge.

A graph is represented as:

G = (V, E)

Where:

V: Set of vertices representing computers.

E: Set of edges representing connections between computers.

Graphs are widely used in computer networks, social networks, navigation systems, and communication systems.

Depth First Search (DFS)

Depth First Search is a graph traversal algorithm that explores a graph by visiting a vertex and then exploring its unvisited adjacent vertices as deeply as possible before backtracking.

DFS can be implemented using recursion or a stack.

Algorithms :

Step 1: Start the program.

Step 2: Create a graph using an adjacency list.

Step 3: Add the connections between the computers.

Step 4: Select the starting computer.

Step 5: Mark all computers as unvisited.

Step 6: Apply DFS to the selected computer.

Step 7: Mark the current computer as visited and display it.

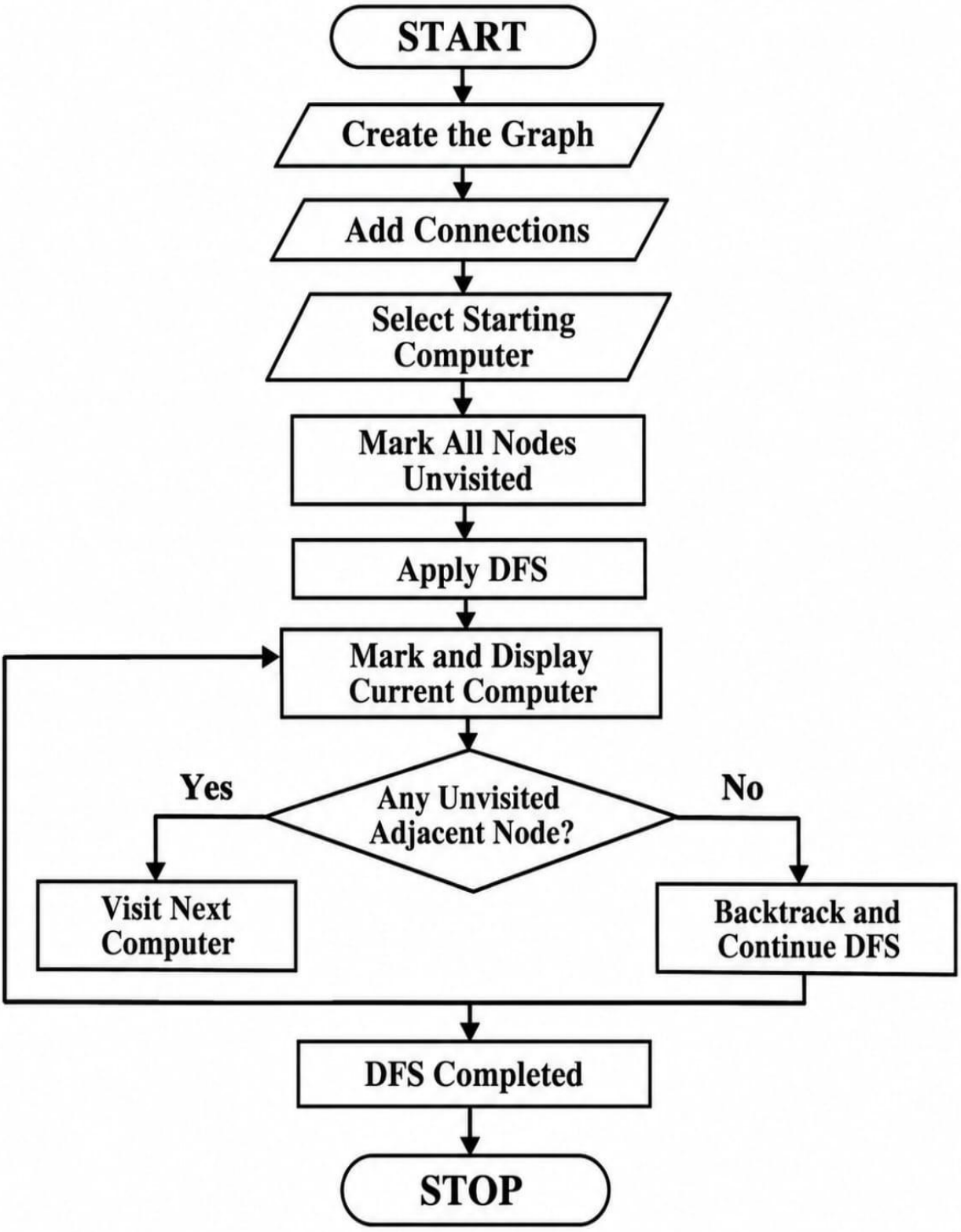
Step 8: Visit each adjacent computer that has not yet been visited.

Step 9: Continue the process until all reachable computers have been visited.

Step 10: Display the DFS traversal representing the reachable computers.

Step 11: Stop the program.

Flowchart :



Program :

```
#include <iostream>
#include <vector>
using namespace std;

void DFS(int node, vector<int> adj[], bool visited[])
{
   visited[node] = true;
   cout << node << " ";

   for (int neighbour : adj[node])
   {
      if (!visited[neighbour])
      {
         DFS(neighbour, adj, visited);
      }
   }
}

int main()
{
   int vertices = 6;

   vector<int> adj[6];

   // Connections between computers
   adj[0].push_back(1);
   adj[0].push_back(2);

   adj[1].push_back(0);
   adj[1].push_back(3);
   adj[1].push_back(4);

   adj[2].push_back(0);
```

```
    adj[2].push_back(4);

    adj[3].push_back(1);
    adj[4].push_back(1);
    adj[4].push_back(2);

    // Computer 5 is disconnected

    bool visited[6] = {false};

    int start;

    cout << "Computer Network DFS Traversal" << endl;
    cout << "Computers are numbered from 0 to 5." << endl;
    cout << "Enter starting computer: ";
    cin >> start;

    if (start < 0 || start >= vertices)
    {
        cout << "Invalid computer number." << endl;
        return 0;
    }

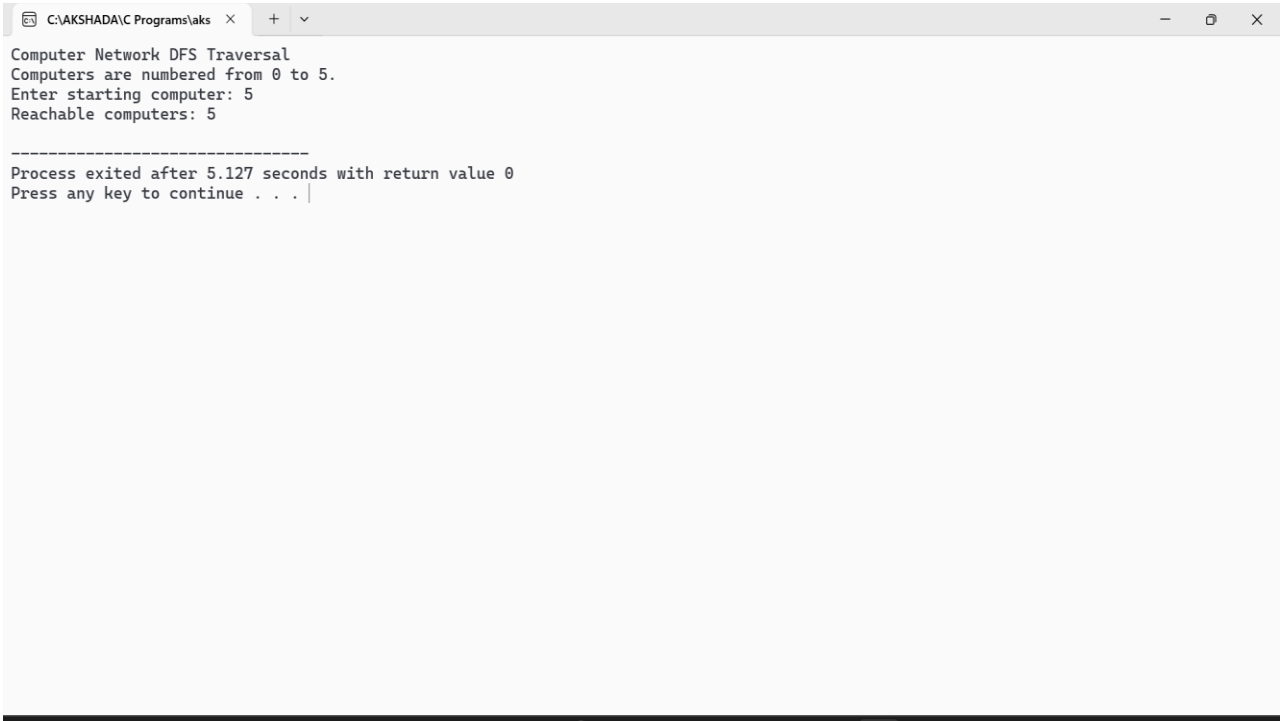
    cout << "Reachable computers: ";

    DFS(start, adj, visited);

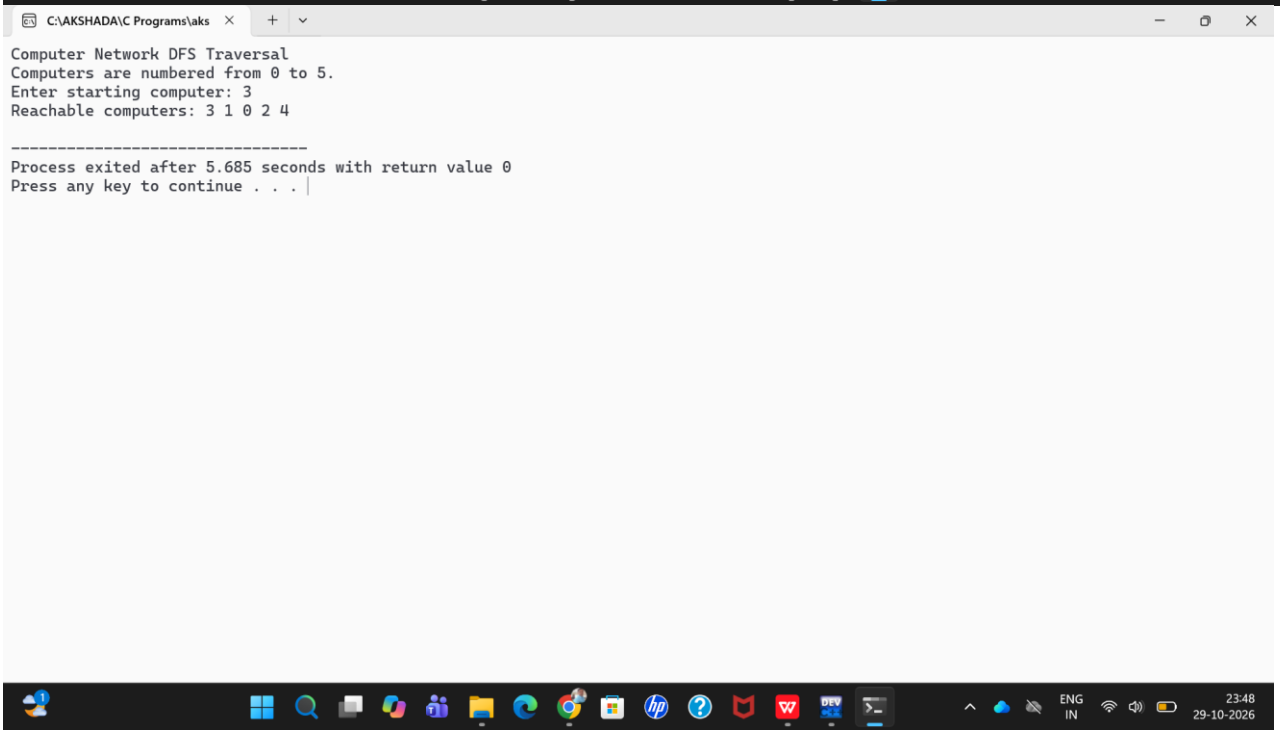
    cout << endl;

    return 0;
}
```

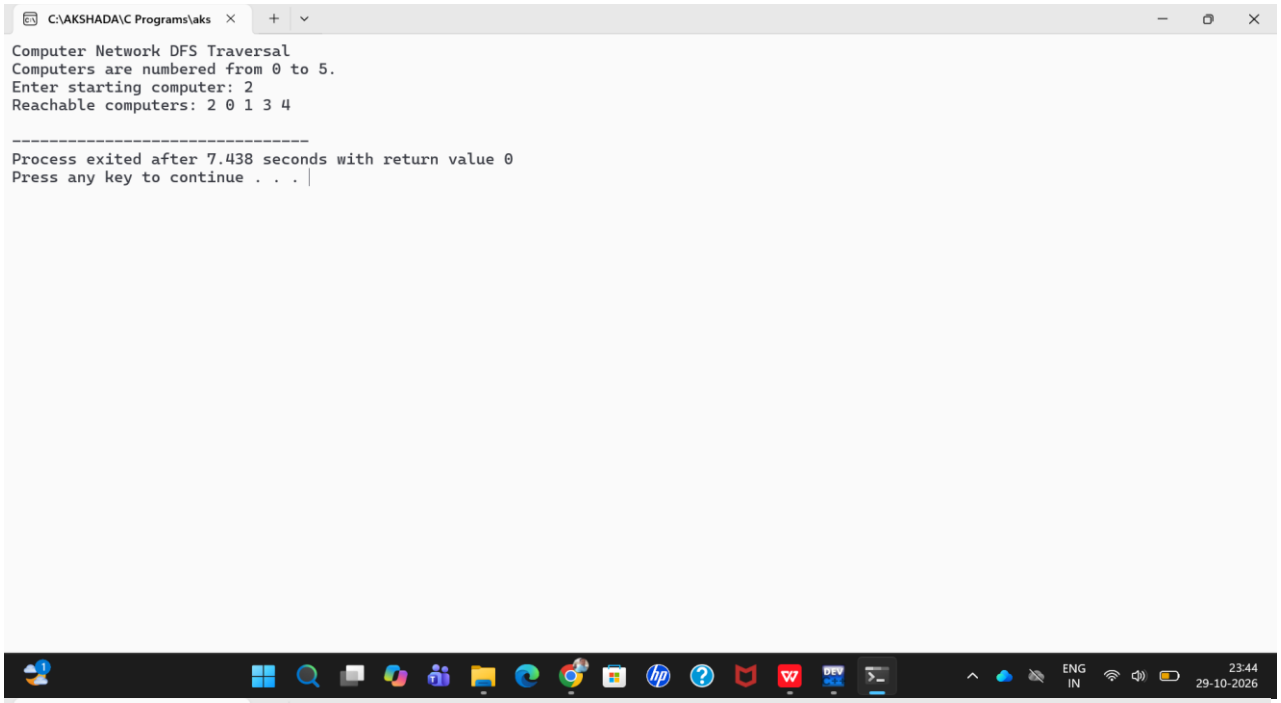
Output :


C:\AKSHADA\C Programs\aks
Computer Network DFS Traversal
Computers are numbered from 0 to 5.
Enter starting computer: 5
Reachable computers: 5

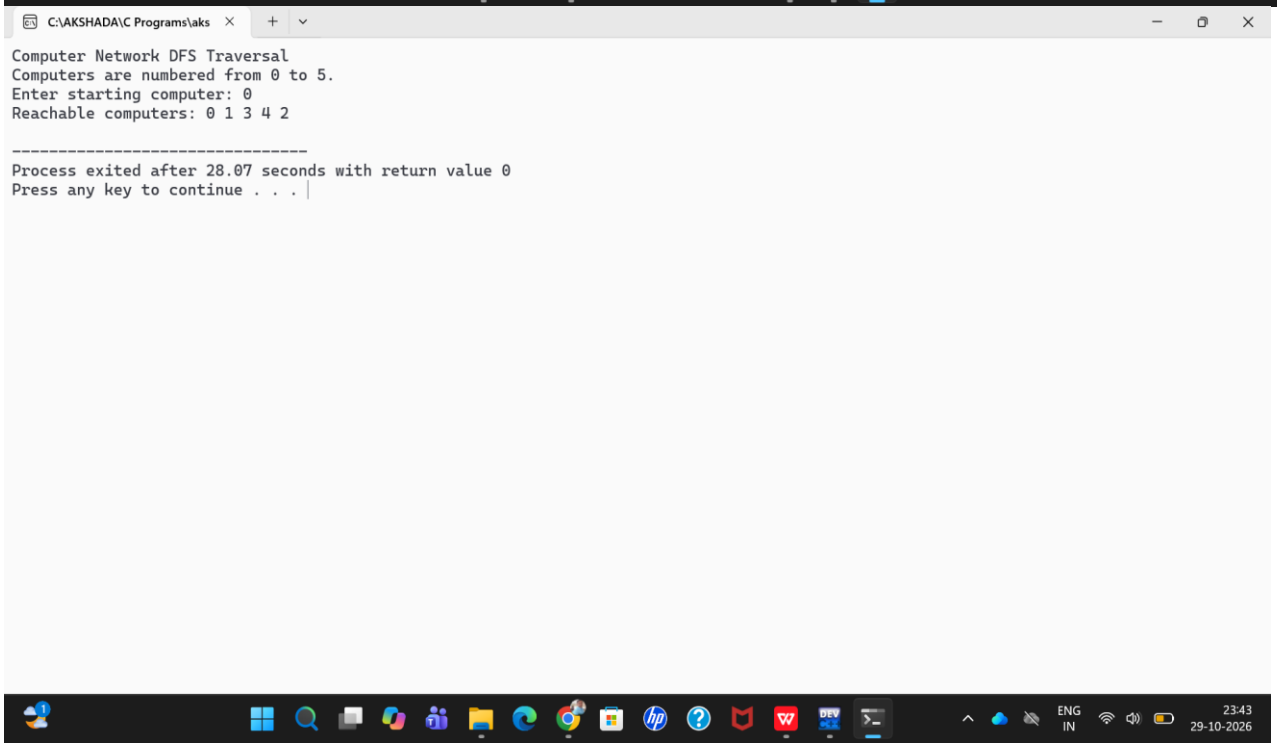
Process exited after 5.127 seconds with return value 0
Press any key to continue . . .
ENG
IN
23:50
29-10-2026


C:\AKSHADA\C Programs\aks
Computer Network DFS Traversal
Computers are numbered from 0 to 5.
Enter starting computer: 3
Reachable computers: 3 1 0 2 4

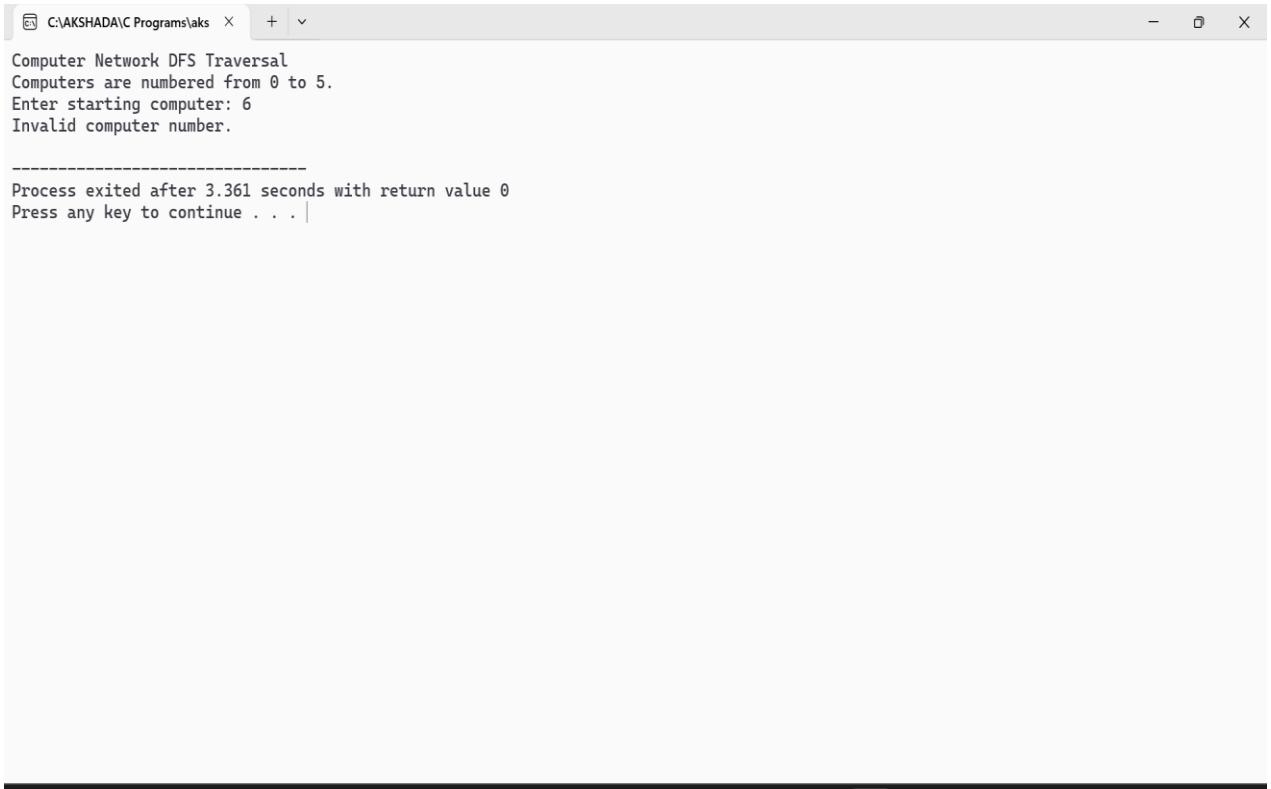
Process exited after 5.685 seconds with return value 0
Press any key to continue . . .
ENG
IN
23:48
29-10-2026


C:\AKSHADA\C Programs\aks
Computer Network DFS Traversal
Computers are numbered from 0 to 5.
Enter starting computer: 2
Reachable computers: 2 0 1 3 4

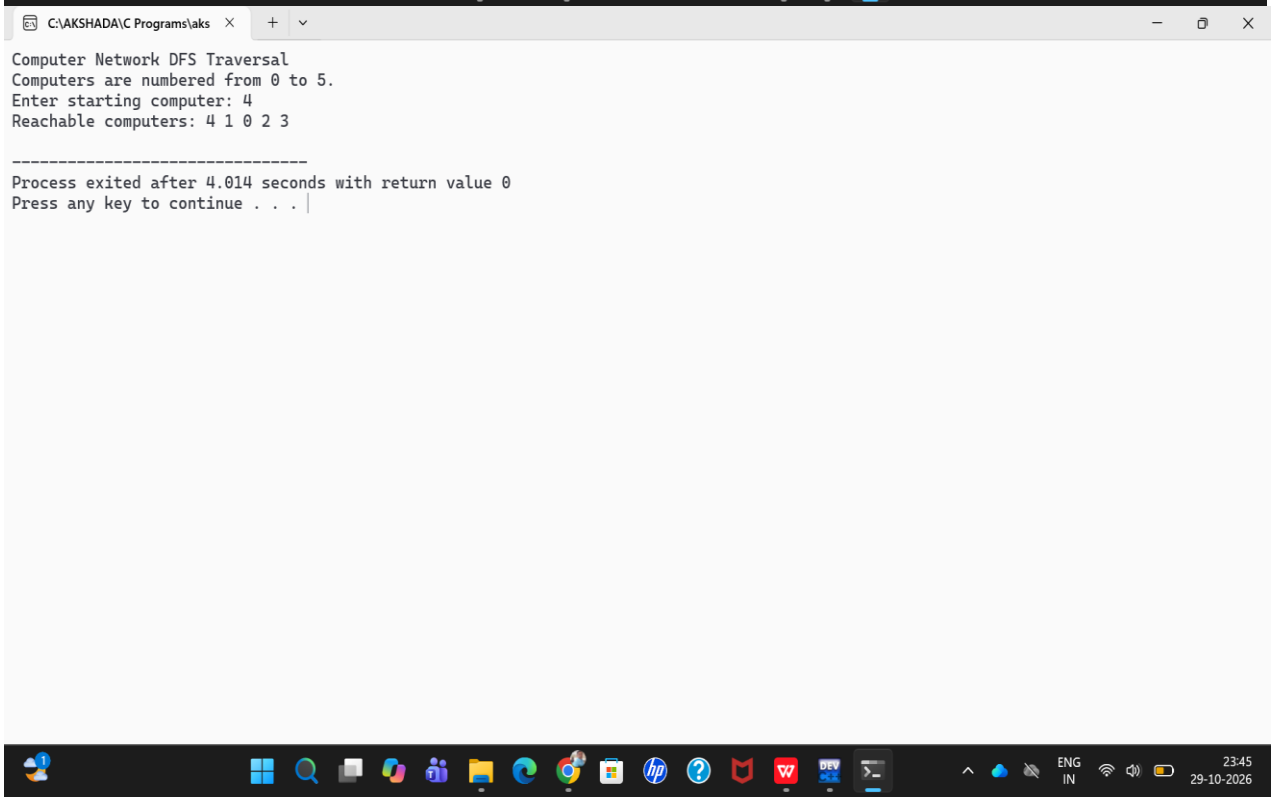
Process exited after 7.438 seconds with return value 0
Press any key to continue . . .
ENG
IN
23:44
29-10-2026


C:\AKSHADA\C Programs\aks
Computer Network DFS Traversal
Computers are numbered from 0 to 5.
Enter starting computer: 0
Reachable computers: 0 1 3 4 2

Process exited after 28.07 seconds with return value 0
Press any key to continue . . .
ENG
IN
23:43
29-10-2026


C:\AKSHADA\C Programs\aks
Computer Network DFS Traversal
Computers are numbered from 0 to 5.
Enter starting computer: 6
Invalid computer number.

Process exited after 3.361 seconds with return value 0
Press any key to continue . . .
ENG
IN
23:46
29-10-2026


C:\AKSHADA\C Programs\aks
Computer Network DFS Traversal
Computers are numbered from 0 to 5.
Enter starting computer: 4
Reachable computers: 4 1 0 2 3

Process exited after 4.014 seconds with return value 0
Press any key to continue . . .
ENG
IN
23:45
29-10-2026

Advantages :

1) DFS efficiently explores connected computers.
2) It is simple to implement using recursion.
3) It helps identify computers reachable from a starting computer.
4) It can be used to analyze network connectivity.
5) It avoids repeated visits by maintaining a visited array.

Applications :

1) Computer network connectivity analysis.
2) Network topology exploration.
3) Path exploration in communication networks.
4) Connected-component identification.
5) Graph-based problem solving.

Conclusion :

Thus, the Depth First Search (DFS) algorithm was successfully implemented in C++ to identify all computers reachable from a selected computer in a network graph. The program demonstrates how graph traversal can be used to analyze network connectivity and explore connected computers efficiently.